

IMAGE PROCESSING AND COMPUTER VISION
IMAGE BASIC OPERATION



KHEN MUHAMMAD CAHYO

244107023002

TI-3I

STATE POLYTECHNIC OF MALANG
INFORMATION TECHNOLOGY DEPARTMENT

PRACTICUM – SIMPLE IMAGE OPERATIONS

- **Linier Brightness Transformation**

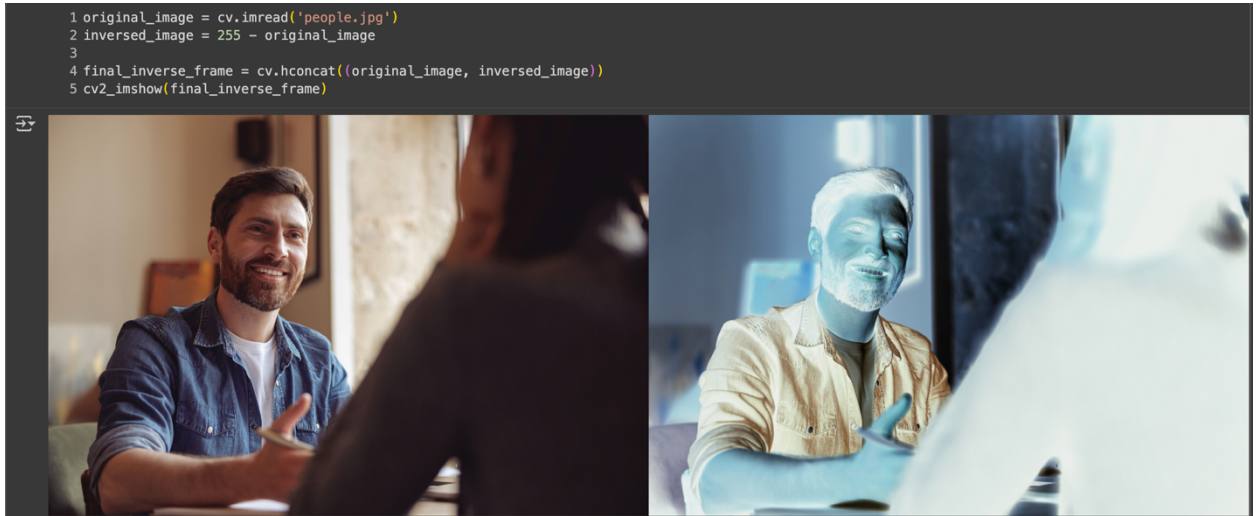
```
1 print('=== Change Brightness of Image ===')
2 print('=====')
3
4 try:
5     brightness = int(input('Input brightness value: '))
6 except ValueError:
7     print('Invalid value')
8     exit()
9
10 original_image = cv.imread('people.jpg')
11 brightness_image = np.zeros(original_image.shape, original_image.dtype)
12
13 # per pixels access
14 # for y in range(original_image.shape[0]):
15 #     for x in range(original_image.shape[1]):
16 #         for c in range(original_image.shape[2]):
17 #             brightness_image[y, x, c] = np.clip(original_image[y, x, c] + brightness, 0, 255)
18
19 # Simple ways without loop
20 brightness_image = cv.convertScaleAbs(original_image, beta=brightness)
21
22 final_lbt_frame = cv.hconcat((original_image, brightness_image))
23 cv2.imshow('final_lbt_frame')
```

```
=== Change Brightness of Image ===
=====
Input brightness value: 50
```



TASK – SIMPLE IMAGE OPERATIONS

- **Number 1 – Inverse Image**

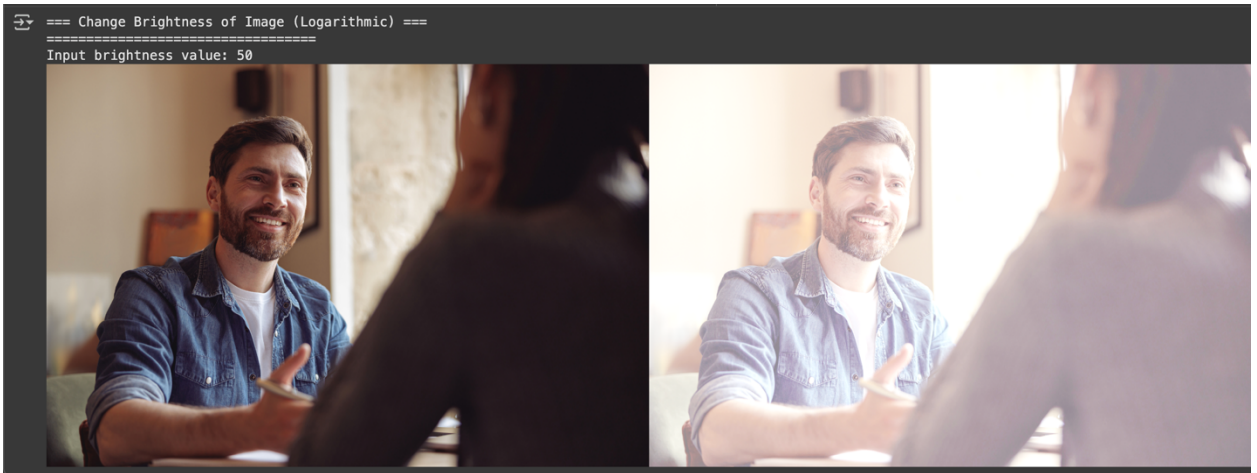


- **Number 2 – Contrast**



- **Number 3 – Logarithmic Brightness**

```
1 original_image = cv.imread('people.jpg')
2 logarithmic_image = np.zeros(original_image.shape, original_image.dtype);
3
4 print('=== Change Brightness of Image (Logarithmic) ===')
5 print('=====')
6
7 try:
8     brightness = int(input('Input brightness value: '))
9 except ValueError:
10     print('Invalid value')
11     exit()
12
13 logarithmic_image = brightness * np.log(1 + original_image.astype(np.float32))
14 logarithmic_image = np.clip(logarithmic_image, 0, 255).astype(np.uint8)
15
16 final_logarithmic_frame = cv.hconcat((original_image, logarithmic_image))
17 cv2_imshow(final_logarithmic_frame)
```



- **Number 4 – Grayscale**

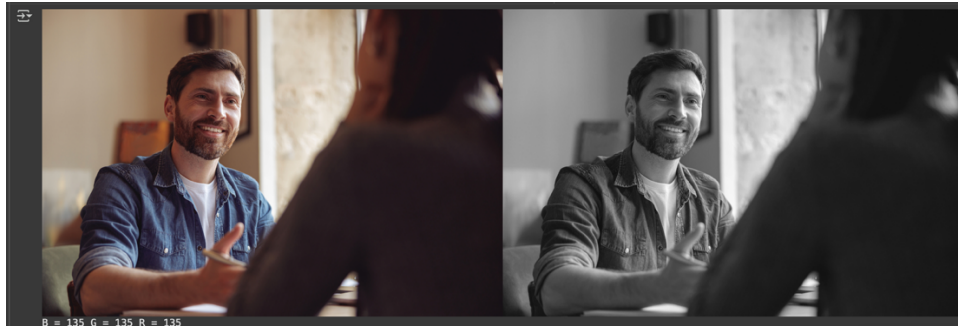
- **Averaging Grayscale**

```
1 # Averaging Grayscale
2 original_image = cv.imread('people.jpg')
3 B, G, R = cv.split(original_image)
4 ag_image = np.zeros(original_image.shape, original_image.dtype);
5
6 ag_image = ((R.astype(np.float32) + G.astype(np.float32) + B.astype(np.float32)) / 3)
7 ag_image = np.clip(ag_image, 0, 255).astype(np.uint8)
8
9 ag_image_bgr = cv.cvtColor(ag_image, cv.COLOR_GRAY2BGR)
10
11 final_ag_frame = cv.hconcat((original_image, ag_image_bgr))
12 cv2_imshow(final_ag_frame)
13
14 (b, g, r) = ag_image_bgr[10, 20] # Get the pixels value on the row 10 column 20
15 print("B =", b, "G =", g, "R =", r)
```



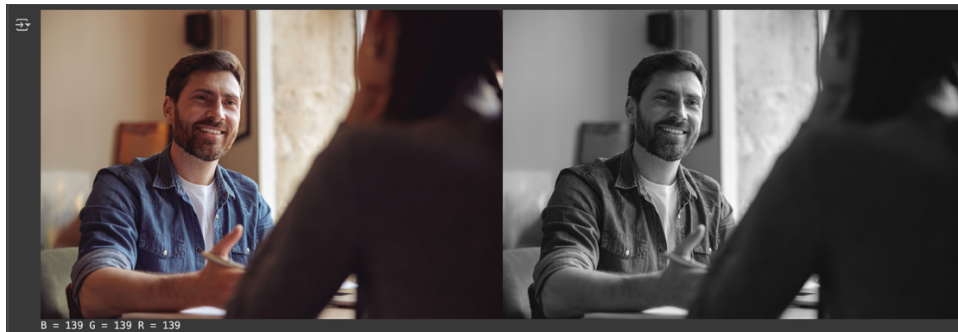
○ Lightness Grayscale

```
1 # Lightness Grayscale
2 original_image = cv.imread('people.jpg')
3 B, G, R = cv.split(original_image)
4 lg_image = np.zeros(original_image.shape, original_image.dtype);
5
6 max_val = np.maximum(np.maximum(R, G), B)
7 min_val = np.minimum(np.minimum(R, G), B)
8
9 lg_image = ((max_val.astype(np.float32) + min_val.astype(np.float32)) / 2)
10 lg_image = np.clip(lg_image, 0, 255).astype(np.uint8)
11
12 lg_image_bgr = cv.cvtColor(lg_image, cv.COLOR_GRAY2BGR)
13
14 final_lg_frame = cv.hconcat((original_image, lg_image_bgr))
15 cv2.imshow('final_lg_frame')
16
17 (b, g, r) = lg_image_bgr[10, 20] # Get the pixels value on the row 10 column 20
18 print("B =", b, "G =", g, "R =", r)
```



○ Luminate Grayscale

```
1 # Luminate Grayscale
2 original_image = cv.imread('people.jpg')
3 B, G, R = cv.split(original_image)
4 lum_image = np.zeros(original_image.shape, original_image.dtype);
5
6 lum_image = (0.21 * R.astype(np.float32)) + (0.72 * G.astype(np.float32)) + (0.07 * B.astype(np.float32))
7 lum_image = np.clip(lum_image, 0, 255).astype(np.uint8)
8
9 lum_image_bgr = cv.cvtColor(lum_image, cv.COLOR_GRAY2BGR)
10
11 final_lum_frame = cv.hconcat((original_image, lum_image_bgr))
12 cv2.imshow('final_lum_frame')
13
14 (b, g, r) = lum_image_bgr[10, 20] # Get the pixels value on the row 10 column 20
15 print("B =", b, "G =", g, "R =", r)
```



- **Number 5 – Convert other colors to Grayscale**

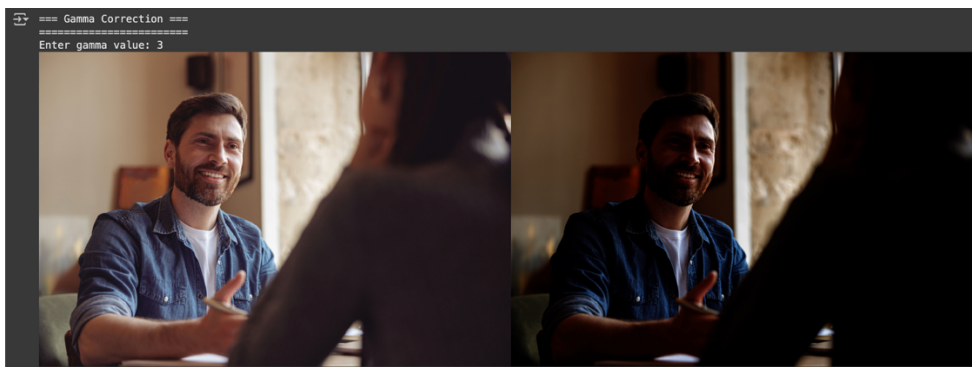
```
1 original_image = cv.imread('people.jpg')
2 converted_to_gray_image = original_image.copy()
3 hsv = cv.cvtColor(original_image, cv.COLOR_BGR2HSV)
4
5 lower_blue = np.array([100, 50, 50])
6 upper_blue = np.array([140, 255, 255])
7
8 mask = cv.inRange(hsv, lower_blue, upper_blue)
9 blue_part = cv.bitwise_and(original_image, original_image, mask=mask)
10
11 gray = cv.cvtColor(converted_to_gray_image, cv.COLOR_BGR2GRAY)
12 gray_bgr = cv.cvtColor(gray, cv.COLOR_GRAY2BGR)
13 gray_bgr_converted = np.where(blue_part != 0, original_image, gray_bgr)
14
15 final_cvt_frame = cv.hconcat((original_image, gray_bgr_converted))
16 cv2.imshow('final_cvt_frame')
```



TASK – ARITHMETIC AND LOGICAL OPERATIONS

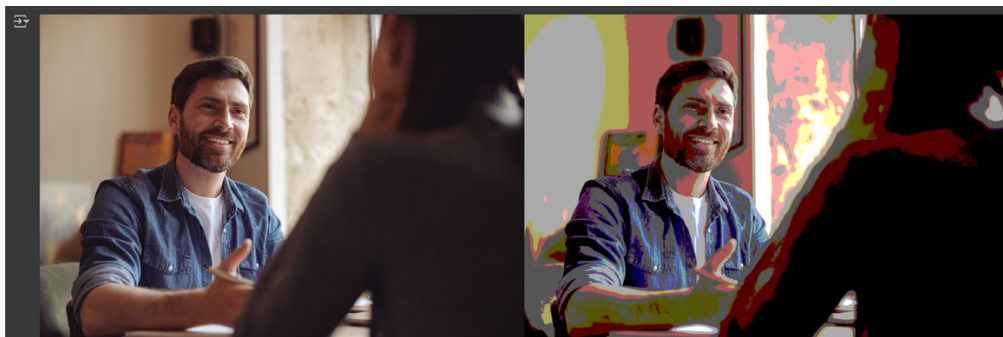
- **Number 1 – Gamma Correction**

```
1 print('=== Gamma Correction ===')
2 print('=====')
3
4 try:
5     gamma = float(input('Enter gamma value: '))
6 except ValueError:
7     print('Invalid value')
8
9 def adjust_gamma(image, gamma=1.0):
10     table = np.array([(i / 255.0) ** gamma * 255 for i in np.arange(256)]).astype("uint8")
11     return cv.LUT(image, table)
12
13 original_image = cv.imread('people.jpg')
14 gamma_image = adjust_gamma(original_image, gamma)
15
16 final_gamma_frame = cv.hconcat((original_image, gamma_image))
17 cv2.imshow('final_gamma_frame')
```



- **Number 2 – Image Depth**

```
1 bit_depth = 2
2 levels = 2 ** bit_depth
3 step = 255 / (levels - 1)
4
5 original_image = cv.imread('people.jpg')
6
7 depth_image = np.round(original_image / step) * step
8 depth_image = np.clip(depth_image, 0, 255).astype(np.uint8)
9
10 final_depth_frame = cv.hconcat((original_image, depth_image))
11 cv2.imshow('final_depth_frame')
```



- **Number 3 – Average Denoising**

- **Codes**

```

1 # Mount the images
2 folder_path = "/content/drive/MyDrive/Collage/Semester 5/PCVK/Dataset/noises/"
3 image_paths = sorted(glob.glob(os.path.join(folder_path, "*.jpg")))
4
5 print('Total of found images: ', len(image_paths))
6
7 # Define PSNR function
8 def PSNR(img1, img2):
9     mse = np.mean((img1 - img2) ** 2)
10
11     if (mse == 0):
12         return 100
13
14     max_pixel = 255.0
15     psnr = 20 * np.log10(max_pixel / np.sqrt(mse))
16     return psnr

```

```

1 # Image Execution
2 selected_paths = image_paths[0:10] # Just change 10 with needed number based on total images
3 images = [cv.imread(p) for p in selected_paths]
4
5 denoised_image = np.mean(images, axis=0).astype(np.uint8)
6
7 # PSNR Evaluation
8 reference = images[0]
9 psnr_before = PSNR(reference, images[1]) # before denoising (noisy vs other noisy)
10 psnr_after = PSNR(reference, denoised_image) # after denoising
11
12 final_denoised_frame = cv.hconcat((reference, denoised_image))
13 cv2.imshow('final_denoised_frame')
14
15 print(f"PSNR before denoising: {psnr_before:.2f} dB")
16 print(f"PSNR after denoising: {psnr_after:.2f} dB")

```

- **Table**

No	Number of Images in AVG	Image & PSNR (dB) Results
1.	10	 PSNR before denoising: 28.43 dB PSNR after denoising: 29.01 dB
2.	20	 PSNR before denoising: 28.43 dB PSNR after denoising: 28.97 dB

3.	40	
4.	80	
5.	100	

○ Conclusions

Based on the results of image denoising tests using the temporal averaging method with different numbers of images (10, 20, 40, 80, and 100), it can be observed that the noise level in the resulting images decreases visually as the number of images used in the smoothing process increases. This is consistent with the theory that the smoothing process (averaging) suppresses random variations in the image, thereby reducing noise.

However, quantitative measurements using the Peak Signal-to-Noise Ratio (PSNR) revealed an interesting phenomenon. When using 10 images, the PSNR value was recorded as higher (around 29.01 dB), but when the number of images was increased from 20 to 100, the PSNR value decreased slightly and tended to stabilize in the range of 28.92–28.97 dB. In theory, a higher PSNR value indicates better denoising quality.

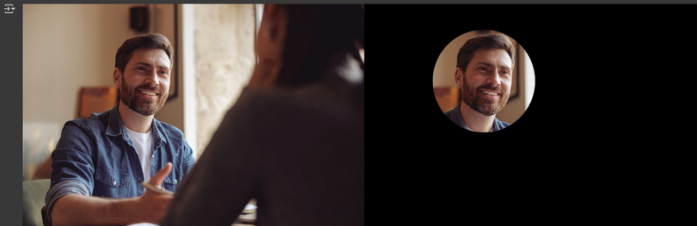
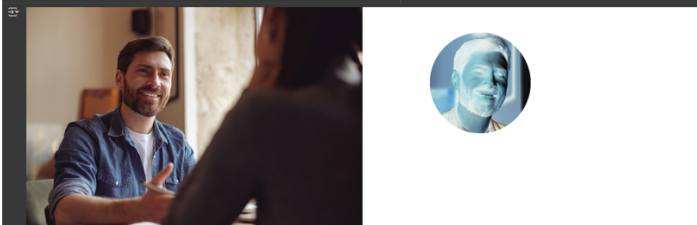
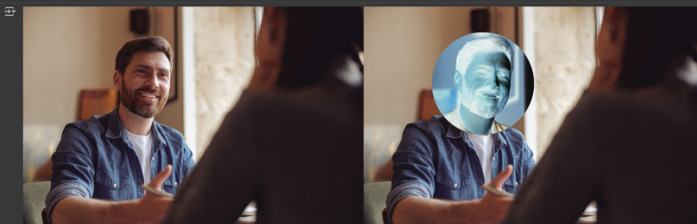
However, this small decrease suggests that increasing the number of images does not necessarily result in a significant improvement in quality.

This phenomenon is likely due to the characteristics of the dataset used. Based on the literature and observations, the images in this dataset have relatively homogeneous noise levels between frames. This means that even if the number of images is increased, the amount of noise that can be "removed" through the averaging process is not significant. Consequently, the PSNR gain obtained is insignificant and tends to plateau after a certain number of images.

Therefore, it can be concluded that although the temporal averaging method is effective in reducing visual noise, increasing the number of images does not always guarantee a significant increase in PSNR, especially if the dataset used has a relatively uniform noise level.

- **Number 4 – Image Masking**

- **Table**

No	Operator	Results I/O
1.	NOT (Complement)	<pre>1 # NOT Operator 2 not_masked_image = cv.bitwise_not(original_image, mask_image) 3 4 frame_not_masked_images = cv.hconcat((original_image, not_masked_image)) 5 cv2.imshow(frame_not_masked_images)</pre> 
2.	OR (Or)	<pre>1 # OR Operator 2 or_masked_image = cv.bitwise_or(original_image, mask_image) 3 4 frame_or_masked_images = cv.hconcat((original_image, or_masked_image)) 5 cv2.imshow(frame_or_masked_images)</pre> 
3.	AND (And)	<pre>1 # AND Operator 2 and_masked_image = cv.bitwise_and(original_image, mask_image) 3 4 frame_and_masked_images = cv.hconcat((original_image, and_masked_image)) 5 cv2.imshow(frame_and_masked_images)</pre> 
4.	NAND (Not And)	<pre>1 # NAND Operator 2 nand_masked_image = cv.bitwise_not(cv.bitwise_and(original_image, mask_image)) 3 4 frame_nand_masked_images = cv.hconcat((original_image, nand_masked_image)) 5 cv2.imshow(frame_nand_masked_images)</pre> 
5.	XOR (Exclusive Or)	<pre>1 # XOR Operator 2 xor_masked_image = cv.bitwise_xor(original_image, mask_image) 3 4 frame_xor_masked_images = cv.hconcat((original_image, xor_masked_image)) 5 cv2.imshow(frame_xor_masked_images)</pre> 

- **Conclusions**

In digital image processing experiments using logical operations (NOT, OR, AND, NAND, XOR) with the help of OpenCV, it was understood that each logical operation has a different effect on the resulting image. The NOT operation reverses pixel intensity values, making bright areas dark and vice versa, similar to the concept of an inverted image. The AND operation indicates that only overlapping areas in both images are retained, while other areas are discarded, making it often used for masking or segmenting specific objects. Conversely, the OR operation combines information from both images, displaying areas containing only one pixel value.

Furthermore, the XOR operation displays differences between images, as only pixels that are not the same in both images are retained, while identical pixels are discarded. This makes XOR often used to detect changes or differences between two images. Meanwhile, the NAND operation, which is the inverse of AND, displays the inversion of overlapping areas, highlighting areas that were previously missed by the AND operation.

Overall, the application of logical operations in image processing can assist in image analysis and manipulation processes, such as segmentation, object extraction, difference detection, and the creation of certain visual effects. Each operation has unique characteristics that can be utilized according to image processing needs.

GitHub Link:

<https://github.com/KhenCahyo13/MyCampus-Jobsheet/blob/smt-5/pcvk/week-3/Week3.ipynb>